

Purpose and stack

Internal IT service desk that returns a structured, policy-grounded Decision for each fixed request or active ticket. The operator UI is static HTML/JS; FastAPI exposes processing, evidence, metrics, and audit routes.

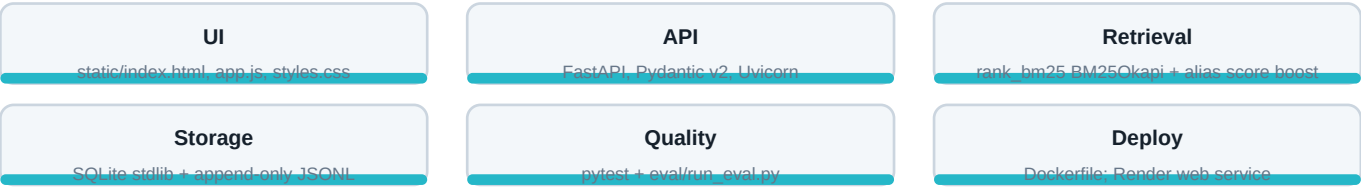
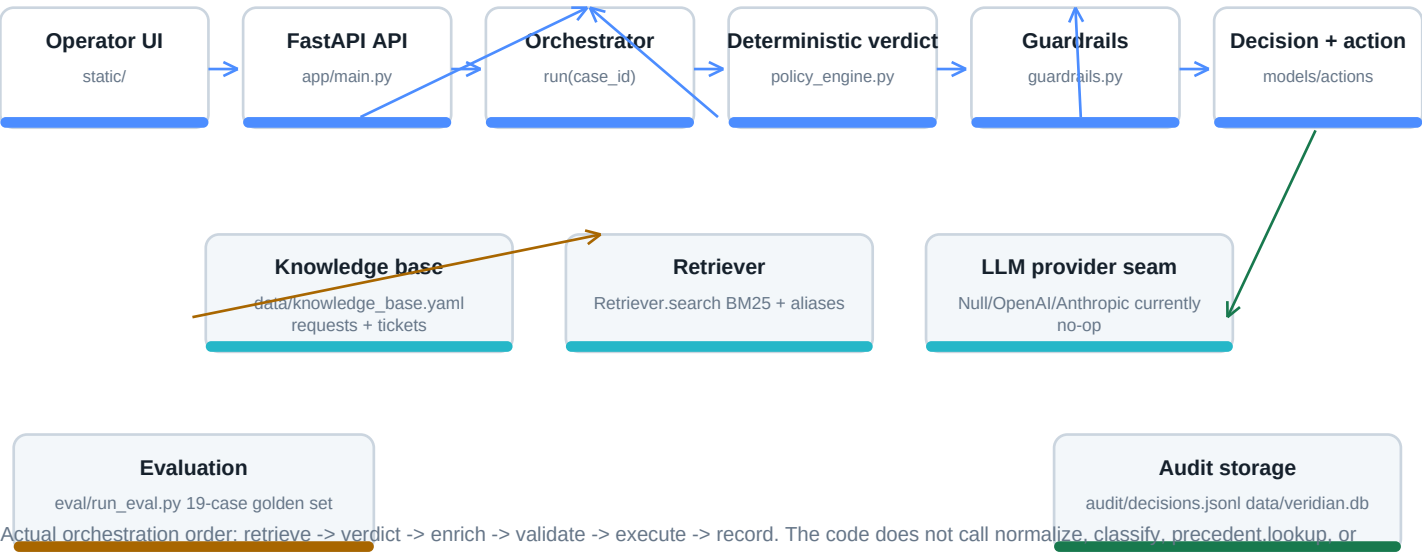


Figure 1. High-level architecture



Actual orchestration order: retrieve -> verdict -> enrich -> validate -> execute -> record. The code does not call normalize, classify, precedent.lookup, or conflict_detector.hardware_conflict in this path.

Component responsibilities

Component	Actual function / input	Output / dependency
app/main.py	HTTP routes; StaticFiles mount	JSON Decision / HTML UI
app/orchestrator.py	Orchestrator.run(case_id)	retrieved chunks, final Decision
app/policy_engine.py	verdict(case) lookup table	preliminary Pydantic Decision
app/reasoner.py	enrich(decision, chunks)	retains deterministic outcome
app/audit.py	record(case, chunks, decision)	JSONL entry + SQLite upsert

Figure 2. Processing pipeline



Sequence: POST /api/run/{id} or GET /api/cases/{id} -> Orchestrator.run -> evidence retrieval -> deterministic decision -> optional no-op enrichment -> validation -> proposed actions -> audit persistence.

Implemented phases

Phase	Input	Processing	Output
1. Case input	case_id	ingest.cases() reads 15 requests plus active tickets	dict with request text and status
2. Retrieve	case['request']	BM25 top-k against KB text/title/aliases; alias bonus	list[dict] chunks + score
3. Deterministic verdict	case dict	policy_engine.verdict selects a hard-coded case rule; SLA helpers calculate date/age	Decision model
4. Optional LLM	Decision + chunks	reasoner calls selected provider. NullProvider returns unchanged; OpenAI/Anthropic inherit it	same Decision
5. Guardrails	Decision	uncited -> route; confidence < .75 -> downgrade; conflict -> route	final Decision
6. Proposed action + audit	final Decision	actions.execute returns action labels; audit.record appends and upserts	JSONL + SQLite

Important code-vs-design distinction

The repository includes classifier.py (classify), precedent.py (lookup), and conflict_detector.py (hardware_conflict). None is called by Orchestrator.run. Normalization is also not implemented as a function. Their stated pipeline roles are therefore proposed/design-only rather than runtime phases.

Relevant dependencies

Dependency	Used by	Role
fastapi / uvicorn	main.py	API and static UI serving
pydantic	models.py	validated Decision schema
rank-bm25	retriever.py	local evidence ranking
sqlite3 (stdlib)	audit.py	decision persistence
pytest / httpx	tests/	API and logic verification

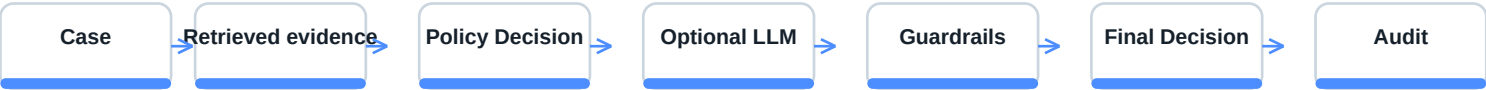
Figure 3. Decision tree - actual final validation behavior



Implemented outcome values

Outcome	Actual use in policy_engine.verdict
AUTO_RESOLVE	Known covered instruction or queue action, e.g. guest Wi-Fi, manual unlock, VPN renewal, archive, repair, or Security review queue.
NEEDS_INFO	Required field/approval/account evidence missing; low confidence case REQ-15 remains NEEDS_INFO after guardrail.
ROUTE_TO_HUMAN	Policy conflict, missing authority, or no policy coverage; REQ-01 and REQ-10 are examples.
FLAG_RISK	Active security violation; REQ-08 produces immediate phishing containment instructions.
MONITOR	Active case already with appropriate owner and within defined SLA / fulfilment status.

Figure 4. Data flow and guardrail controls



Storage: audit.record writes JSONL (append-only) and SQLite decisions(case_id PRIMARY KEY, payload, created_at). Guardrails implemented in code: grounding, confidence, conflict, and audit. There is no separate authority guard function; authority is encoded in verdict rules and owner values.

A. Important project structure

Path	Actual responsibility
app/	FastAPI app, models, orchestration, rule engine, retrieval, audit, LLM provider seam.
data/	knowledge_base.yaml stored as JSON; requests/tickets; golden_set; SQLite created at runtime.
static/	Single-page operator console: HTML, CSS, JavaScript.
eval/ and tests/	No-LLM golden-set evaluation; pytest engine and API checks.
scripts/ and docs/	Asset/deck/PDF generators and submitted images/deck.
Dockerfile, render.yaml	Container startup and Render web-service configuration.

B. Actual API

Method	Path	Purpose
GET	/healthz	Returns ok, frozen clock, deterministic mode.
GET	/api/cases	Returns active case inputs.
GET	/api/kb	Returns locally stored policy records.
GET	/api/cases/{id}	Processes one case and returns Decision; 404 if absent.
POST	/api/run and /api/run/{id}	Processes all cases or a specified case.
POST	/api/ask	KB-only answer; score < 3 returns not covered by policy.
GET	/api/metrics and /api/audit	Runs metric decisions; returns last 100 audit JSONL rows.

C. Real walkthrough: REQ-08

Input: Ananya Reddy reports a suspected phishing email and says it is being forwarded to teammates. The case is read from data/requests.json; actual runtime classification is embedded as category='security' in policy_engine.verdict, not produced by classifier.py.

Evidence KB-09, TK-1048	Rule logic Forwarding violates KB-09	Initial decision FLAG_RISK, owner IT Security
Guardrails Citations exist; 0.99 confidence; no conflict override	Action Issue containment instruction; preserve Security escalation	Audit Case + retrieved IDs + trace + Decision to JSONL/SQLite

Why this final decision is safe

The deterministic table selects the incident response. The draft reply tells the employee to stop forwarding, do not click links, recall/delete copies, and report Security. The action remains proposed only: actions.execute returns labels and does not call external systems. The final audited decision is FLAG_RISK.

Deployment verification: Docker runs evaluation and batch processing before Uvicorn. Render is configured to run uvicorn app.main:app on \$PORT with /healthz as the health endpoint. LLM_PROVIDER is configured as null in render.yaml.